Programming for Data Science

# Reproducible Workflow

## Conda, Jupyter, Markdown

Phạm Trọng Nghĩa

ptnghia@fit.hcmus.edu.vn

# It works on my machine

- Has this happened to you?
  - A classmate's notebook crashes on your computer
  - Your own notebook from last semester no longer runs
  - You get a different number from the same code
- **Today:** why this happens, and how to prevent it.

# Principle: Reproducibility

# What does "reproducible" mean?

- Someone else (or **you, in 6 months**) can rerun your analysis and get **the same result**.
- Why it matters:
  - **Trust:** results nobody can rerun are hard to believe
  - **Teamwork:** your group can build on each other's work
  - **Your future self:** the collaborator you'll work with most

# The 5 pillars of a reproducible result

| Pillar | Breaks when… | Fix |
|---|---|---|
| *1.* ***Code*** | "I changed something but don't remember what" | Git |
| *2.* ***Data*** | File renamed, moved, or edited by hand | Relative paths, raw data is read-only |
| *3.* ***Environment*** | Different package versions | **Conda** |
| *4.* ***Execution order*** | Cells run out of order | **Restart & Run All** |
| *5.* ***Randomness*** | No random seed | rng = np.random.default_rng(42) |

# How bad is it in practice?

- A study(1) of **~1.4 million** Jupyter notebooks on GitHub found:
  - Only about **1 in 4** ran from top to bottom without errors
  - Only about **1 in 25** reproduced their original results
- Top causes:
  - Missing dependencies
  - Missing data files
  - Out-of-order execution

*(1) Pimentel et al., "A Large-scale Study About Quality and Reproducibility of Jupyter Notebooks", MSR 2019*

# Reproducibility is a spectrum

1. "Trust me, the number is 42“ (Bad)
2. Code is shared
3. Code + data are shared
4. Code + data + **environment** are shared (Good)
5. …and it **runs top to bottom** with one command (Good)

- Goal for your final project: level 5.

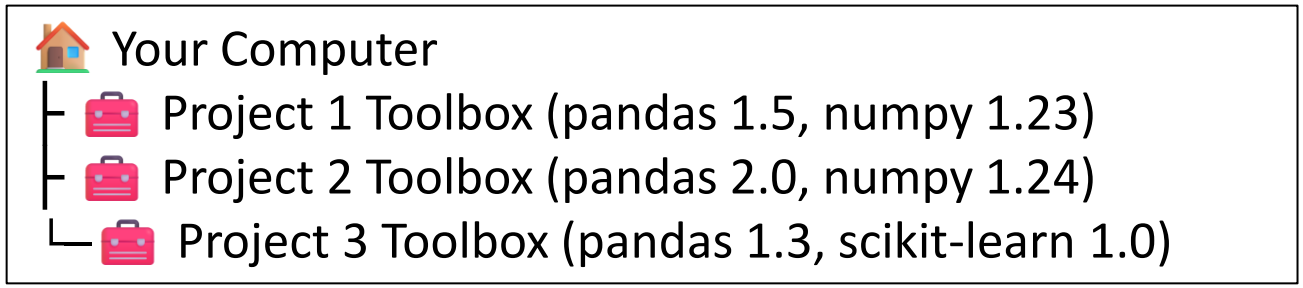
# Some term

- **Package:** a library (pandas) at a specific **version** (2.2.3)
- **(Virtual) Environment:** an isolated folder with one Python + a set of packages
- **Channel:** where packages come from (`conda-forge`, `defaults`)
- **Solver:** finds versions that are all **compatible** with each other

**One project = one environment**

# What is a Virtual Environment?

- **Definition**:
  - A **virtual environment** is an isolated Python workspace where you can install specific versions of packages **without affecting** other projects.
- **Analogy**: Think of it like separate toolboxes for different projects:




- Each project has its own tools (packages) that don't interfere with each other!

# Why Do We Need Virtual Environments?

- **Avoid Conflicts:** different projects need different versions:
  - Project A needs pandas 1.5
  - Project B needs pandas 2.0
  - Can't have both installed globally!
- **Reproducibility:** Share exact environment with teammates:
  - "Here's my code AND my exact setup“
  - Others can recreate your environment perfectly

# Why Do We Need Virtual Environments?

- Clean Workspace
  - Install only what you need per project
  - Easy to delete and start fresh
  - No "leftover" packages from old projects
- Professional Practice
  - Industry standard approach
  - Required for deployment
  - Makes collaboration easier
- **We will use conda for virtual environment in this course**

# Create a New Environment

```
# Basic creation
conda create --name myproject python=3.11
# With specific packages
conda create --name datasci python=3.11 pandas numpy jupyter
# With specific versions
conda create --name ml python=3.10 pandas=2.0 numpy=1.24
```

- What's Happening?
  - Creates isolated environment named "myproject“
  - Installs Python 3.11
  - Installs specified packages
- **Tip**: use descriptive names: covid-analysis, ml-project, thesis-2025

# Create a New Environment

```
# Activate
conda activate myproject
# Your prompt changes to show active environment:
(myproject) $
#...
# Deactivate
conda deactivate
```

- What This Means:
  - All Python commands now use this environment's Python
  - All package installations go into this environment
  - Jupyter notebooks launched here use this environment

# Useful Commands

```
# List all environments
conda env list

# List packages in current environment
conda list

# Update a package
conda update pandas

# Remove a package
conda remove pandas

# Delete an environment
conda env remove --name myproject

# Get help
conda --help
conda create --help
```

# Reproducibility - environment.yml

- **The Problem**: "I want to share my project with a classmate. How do they get the same packages?“
- Using yml file

```
# environment.yml
name: p4ds
channels:
  - conda-forge
dependencies:
  - python=3.11
  - pandas=2.2
  - matplotlib=3.9
  - ipykernel
  - pip
  - pip:
      - some-pip-only-package
```

# Reproducibility - environment.yml

- **Channels**:
  - Locations (repositories) where Conda looks for packages to download and install.
  - Different "app stores" for Python packages.
  - Python packages can exist in different channels, sometimes with different versions or compilation options
- **Your classmate recreates**:

```
conda env create -f environment.yml
```

- **Why Better**?
  - Includes Python version
  - Includes channels
  - More complete information

# How precise should versions be?

| In ***environment.yml*** | **Meaning** |
|---|---|
| *pandas* | Any version (least reproducible) |
| *pandas=2.2* | Any 2.2.x (good default) |
| *pandas==2.2.3* | Exactly this version |

- Trade-off: **stricter = more reproducible, but harder to install** on other machines.

# Conda + pip: the safe order

1. Install everything you can with **conda**
2. Then install the rest with **pip**, inside the activated environment
3. Record pip packages under `- pip:` in `environment.yml`

# Best Practices

- Create environment per project
- Activate before working
- Document your environment
  - Include environment.yml in Git repo
  - Update it when you add packages
- Use descriptive names
  - Good: covid-analysis, ml-homework3
  - Bad: env1, test, temp

# Beyond Conda

| Tool | Environment file |
|---|---|
| **Conda** | *environment.yml* |
| **pip + venv** | *requirements.txt* |
| **uv / Poetry** | *pyproject.toml + lock file* |
| **Docker** | *Dockerfile* |

- Different syntax, **same idea**: the environment is described as text, versioned in Git, rebuilt with one command.

# Jupyter notebook

Introduction

# Why need Integrated Documentation

- Data science workflows naturally alternate between **explanation** and **execution**.
- We need to:
  - Articulate the question we're investigating
  - Write code to explore the data
  - Interpret and document our findings
  - Adjust our approach based on what we learn

# Why need Integrated Documentation

This narrative structure serves two critical purposes

- **Continuity**
  - When returning to work after a break, the documented thought process helps us quickly resume where we left off
- **Communication**
  - Stakeholders can follow our reasoning, verify our methodology, and understand how we arrived at our conclusions

# What is Jupyter Notebook

- **Jupyter Notebook** is an open-source web application that allows you to create and share documents containing live code, equations, visualizations, and narrative text.
- **Key Features:**
  - Interactive computing environment
  - Supports 40+ programming languages (Python, R, Julia, etc.)
  - Combines code execution with rich text, equations, and visualizations
  - Easy sharing and collaboration through .ipynb files
  - Runs in web browser but executes code locally or remotely
- **Origin:**
  - Evolved from IPython in 2014. Name comes from Julia, Python, and R (the three core languages).

# Offers a single document that contains

- Visualizations
- Mathematical equations
- Statistical modeling
- Narrative text
- Any other rich media

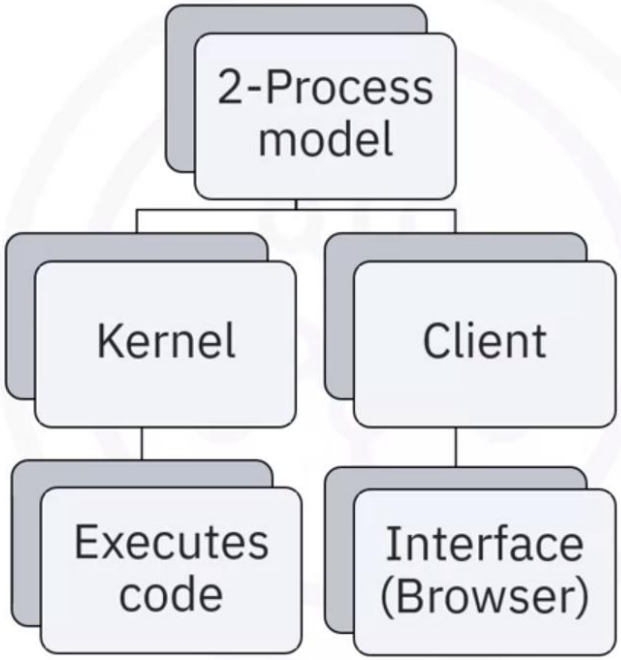
This single document approach enables users to develop, visualize the results and add information, charts, and formulas that make work more **understandable**, **repeatable**, and **shareable**.

# Two variants of the Jupyter notebook

- **Jupyter Classic Notebook**, with all the capabilities mentioned above.
- **JupyterLab**, a new next-generation notebook interface designed to be much more extensible and modular, with support for a wide variety of workflows from data science, machine learning, and scientific computing.

# Jupyter Notebook Architecture

- Jupyter operates as a server-client application that runs in your web browser.
- The **notebook server** handles *code execution* and *file management*, while **your browser** provides the *interactive interface* for editing and running notebooks.

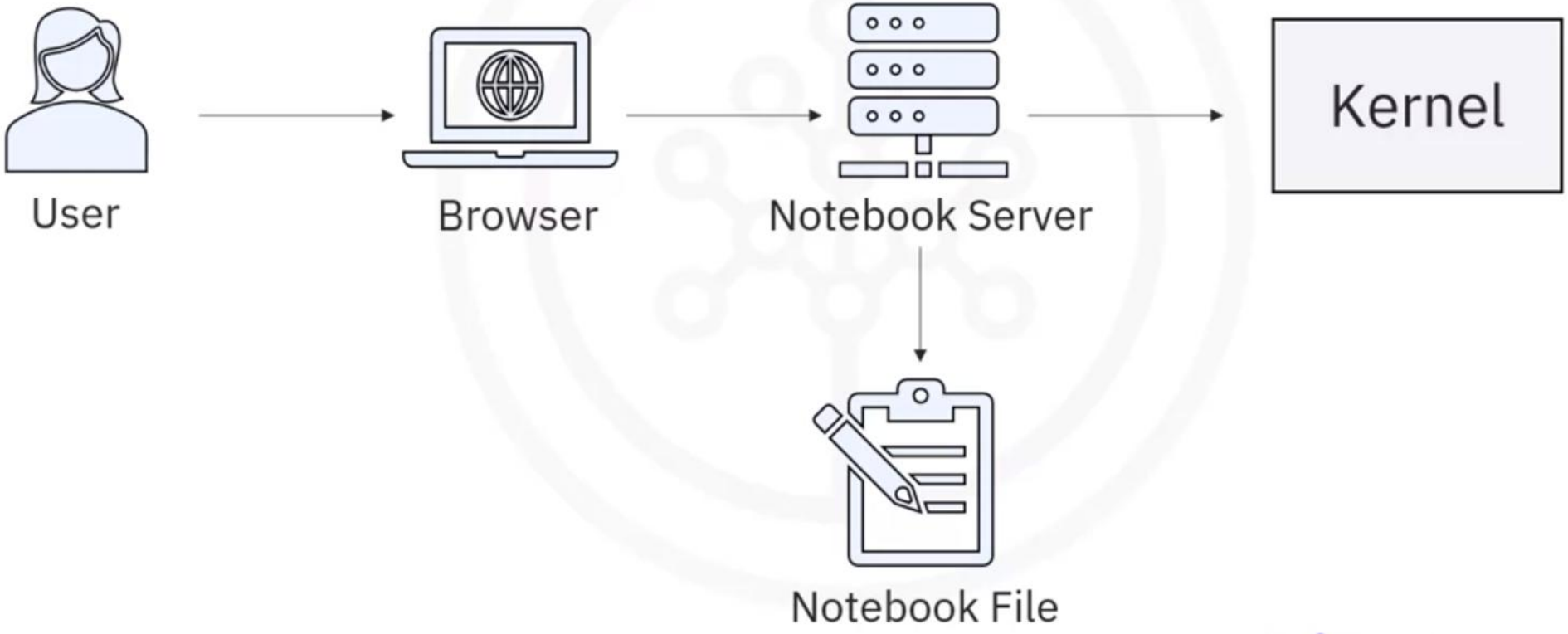



*https://www.coursera.org/learn/open-source-tools-for-data-science/*

# Jupyter Notebook Architecture

- This architecture offers versatility:
  - **Local mode** - Run the server on your own machine and access it through localhost. No internet connection required—everything stays on your computer.
  - **Remote mode** - Install the server on a remote machine (cloud server, institutional cluster, etc.) and access it from anywhere via the internet.
- This enables working with large datasets or computational resources that exceed your local machine's capabilities. Both configurations provide the same notebook experience; only the location of the computational backend changes.

# Jupyter Notebook App



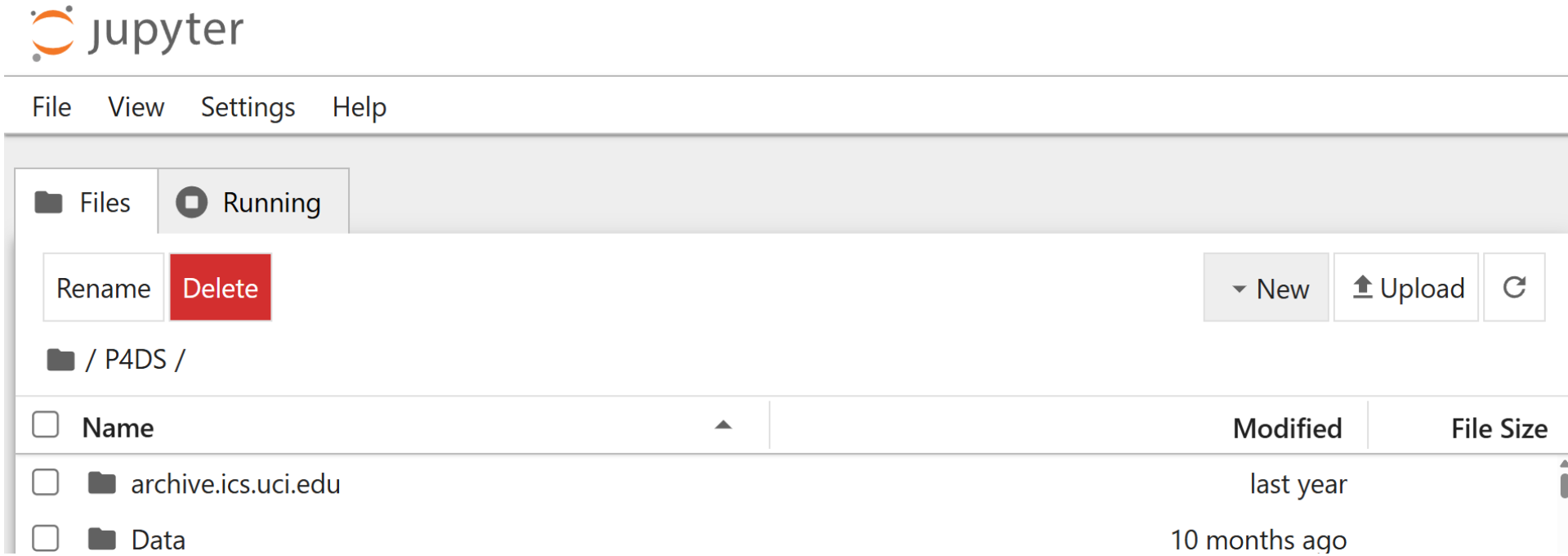
# Jupyter Notebook App

- Its two main components are the kernels and a dashboard.
- **Kernels: The Execution Engine**
- A kernel is the computational engine that executes your code. When you run a cell in a notebook, the kernel:
  - Executes the code in the appropriate programming language
  - Maintains the session state (variables, imports, etc.)
  - Returns output and results back to the notebook
- While Jupyter was originally built for Python (the "Py" in Jupyter), it now supports over 40 languages through different kernels, including R, Julia, JavaScript, and many others.

# Jupyter Notebook App

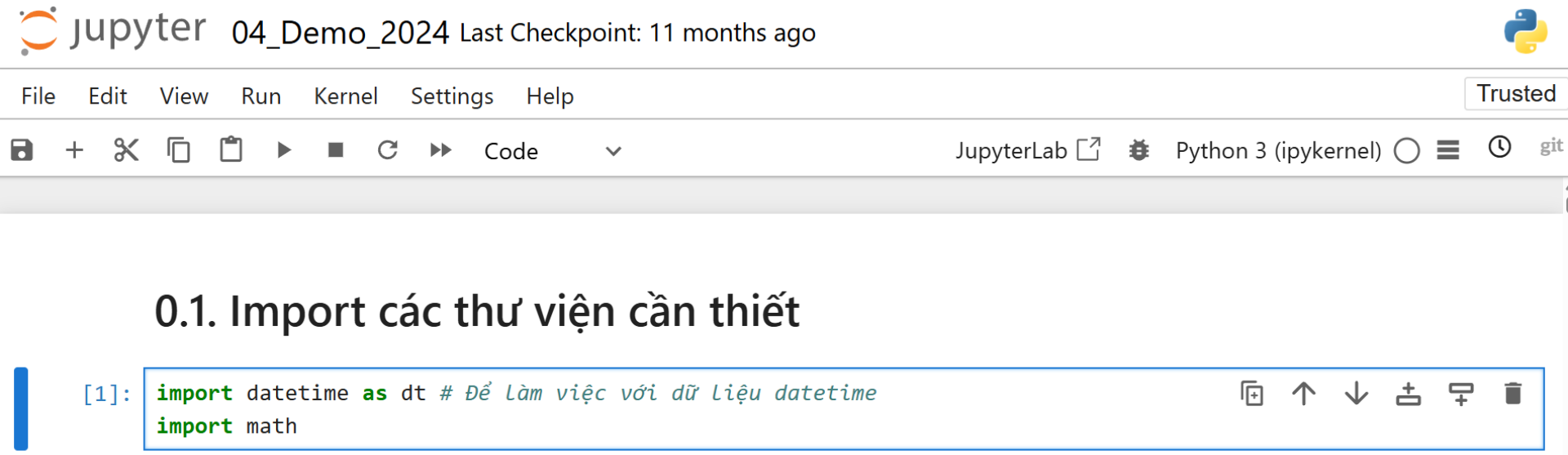
- Dashboard: The Control Center
- The Jupyter dashboard serves as your workspace management interface. From here, you can:
  - Browse and open existing notebook files
  - Create new notebooks
  - Monitor active kernels and see which notebooks are currently running
  - Shut down kernels to free up memory and computational resources when they're no longer needed
  - This separation between the notebook interface and kernel management allows you to efficiently handle multiple notebooks and control resource usage.

# Jupyter Notebook Interface



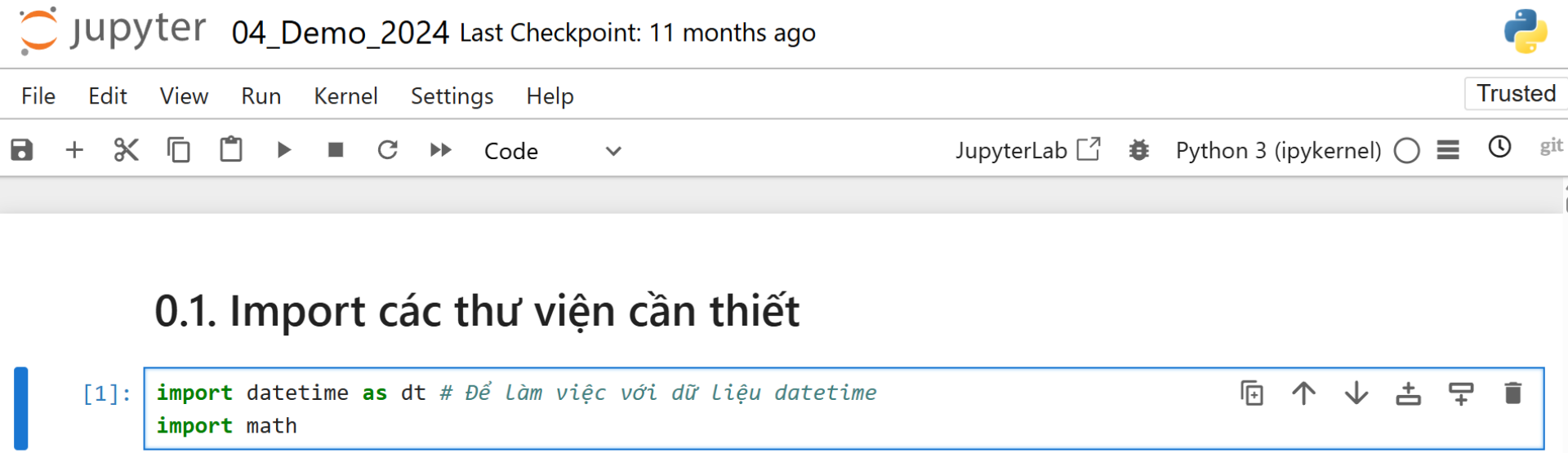

- **Dashboard** (Home Page):
  - File browser showing directory structure
  - Running notebooks and terminals
  - Create new notebooks, text files, folders

# Jupyter Notebook Interface




- **Notebook Interface Components:**
  - **Menu Bar:** File, Edit, View, Insert, Cell, Kernel, Help
  - **Toolbar:** Quick access buttons for common operations
  - **Cells:** Individual units containing code or text

# Jupyter Notebook Interface

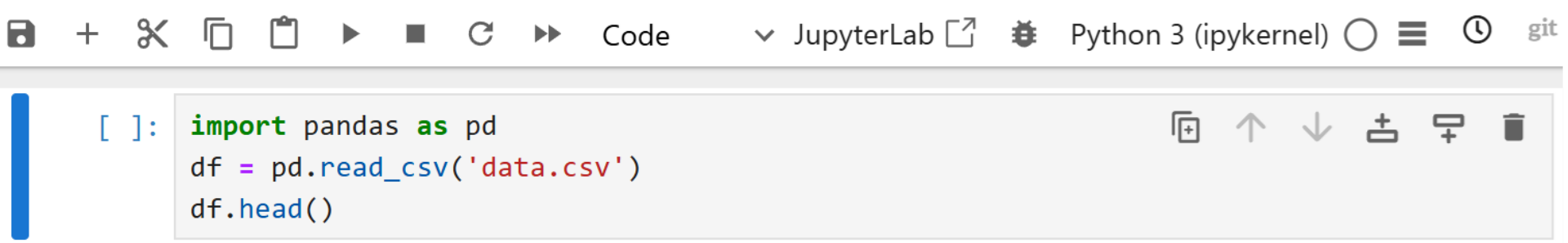



- **Two Modes:**
  - **Edit Mode:** Type into cells
  - **Command Mode:** Navigate and manipulate cells

# Types of Cells

## 1. Code Cells

Execute programming code (Python, R, etc.)

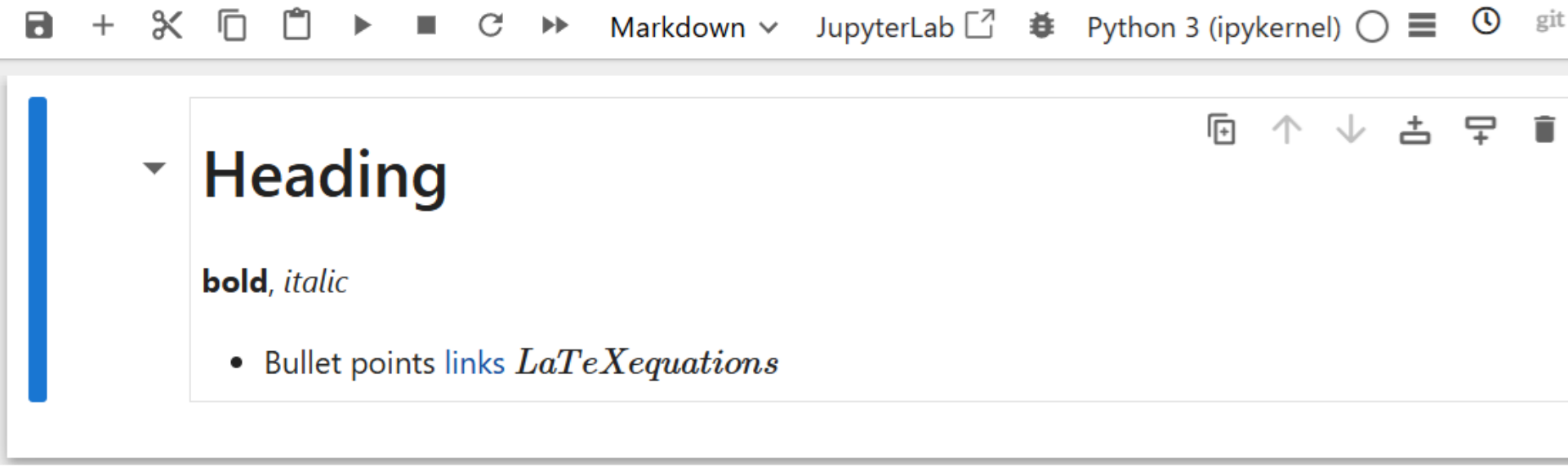



- Output appears directly below the cell
- Can display text, tables, plots, etc.
- Execution count appears in brackets: In [1]

# Types of Cells

## 2. Markdown Cells

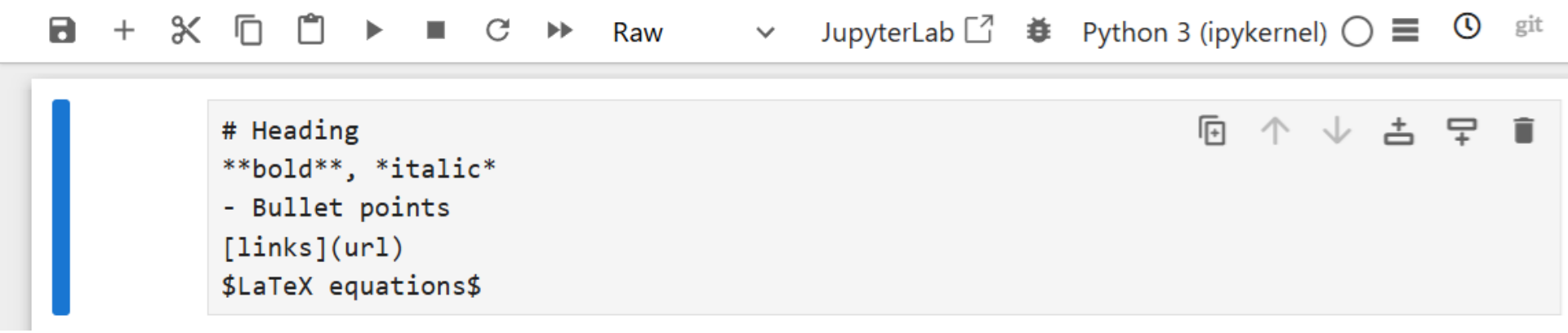
Formatted text using Markdown syntax



# Types of Cells

### 3. Raw Cells

Unformatted text that won't be executed or rendered. Used for special conversions.



# Jupyter notebook reference

- https://jupyter-notebook.readthedocs.io/en/stable/notebook.html
- https://www.edureka.co/blog/wp-content/uploads/2018/10/Jupyter_Notebook_CheatSheet_Edureka.pdf

# Essential Keyboard Shortcuts

**Command Mode (Press Esc to enter):**

- **A** - Insert cell above
- **B** - Insert cell below
- **D, D** - Delete cell (press D twice)
- **M** - Convert to Markdown cell
- **Y** - Convert to Code cell
- **Z** - Undo cell deletion
- **Shift + Up/Down** - Select multiple cells

# Essential Keyboard Shortcuts

**Edit Mode (Press Enter to enter):**

- **Ctrl + Enter** - Run cell
- **Shift + Enter** - Run cell and select below
- **Alt + Enter** - Run cell and insert below
- **Tab** - Code completion or indent
- **Shift + Tab** - Tooltip (function documentation)

# Working with Code Cells

- **Magic Commands:**
- Special commands prefixed with % (line magic) or %% (cell magic)

```
%matplotlib inline  # Display plots inline
%lsmagic            # List the Magic Commands
%timeit func()      # Time code execution
%%time              # Time entire cell
%load file.py       # Load external code
%run script.py      # Execute external script
%who                # List variables
%pwd                # Print working directory
```

# Learn From the Best Notebooks

- Notebooks are also used to complement books, such as the Python Data Science Handbook. You can find the notebooks here.
- This matplotlib tutorial is an excellent example of how well a notebook can serve as a means of teaching other people topics such as scientific Python.

# Jupyter notebook

Principle: *the notebook should mean what it shows*

# How Jupyter really works

- The notebook (`.ipynb`) is just a **document**: cells + saved outputs
- The **kernel** is a separate running Python process holding all variables
- Running a cell = sending its code to the kernel

The kernel remembers **everything you ran, in the order you ran it**, not the order shown on screen.

# What's wrong with this notebook?

```
In [1]:  1  price = 100

In [3]:  1  price = price + 10

In [4]:  1  total = price * 3

In [5]:  1  print(total)

         360
```

# Restart & Run All

- **Kernel → Restart Kernel and Run All Cells**
  - Clears memory, runs every cell top to bottom
  - If it passes, the notebook means what it shows
  - Do it **before every commit and every submission**

# Prevent hidden state

- **Imports** and **settings** in the first cell only
- **Don't overwrite** raw data:
  - `df_clean = clean(df_raw)` ☑
  - `df = clean(df)` ☒
- **Avoid** in-place updates like `x = x + 1` across cells
- Use **relative paths**:
  - `data/sales.csv` ☑
  - `C:/Users/xyz/Desktop/sales.csv` ☒
- **Seed** randomness once at the top
- **Restart often**: it's cheap, bugs are expensive

# Notebook or .py file?

- **Notebook**:
  - Exploring, trying ideas
  - Showing results with a story
  - Charts, tables, narrative
- **Python module (.py)**
  - Functions you reuse
  - Code other notebooks import
  - Code you want to test

# Literate programming & Markdown

Principle: code + story, together

# Markdown

- Markdown is a lightweight markup language with plain text formatting syntax designed to be converted to HTML and many other formats.
- **Key Characteristics:**
  - Easy to read and write in plain text
  - Minimal syntax that doesn't obscure content
  - Can be converted to HTML, PDF, and other formats
  - Platform-independent and future-proof
  - Widely supported across applications and websites

# Markdown

- Markdown allow us to write plain text with marks to specify simple and widely used formats such as heading, bullet, italic, bold, …
- It's **easy to read and write**
- Restricting users to a small set of formats also helps users to **focus more on content** when writing

# Why Use Markdown - Advantages

- **Simple**: Learn the basics in 10 minutes
- **Fast**: Write formatted text without leaving keyboard
- **Portable**: Plain text files work everywhere
- **Version Control Friendly**: Perfect for Git/GitHub
- **Focus on Content**: Formatting doesn't distract from writing
- **Universal**: Supported by countless platforms and tools

# Common Use Cases

- Documentation (README files, wikis)
- Note-taking and knowledge management
- Blog posts and articles
- GitHub issues and pull requests
- **Jupyter Notebooks**
- Chat messages (Slack, Discord, Teams)

# Markdown - Reference

- https://www.markdownguide.org/basic-syntax/
- https://www.markdowntutorial.com/

# Headers

- **Using Hash Symbols (#)**

```
# Header 1
## Header 2
### Header 3
#### Header 4
##### Header 5
###### Header 6
```

- **Alternative Syntax (H1 and H2 only)**

```
Header 1
========

Header 2
--------
```

# Headers

- **Best Practices:**
  - Add a space after the # symbol
  - Use headers hierarchically (don't skip levels)
  - Blank lines before and after headers improve readability

# Text Formatting

- **Emphasis**

```
*italic* or _italic_
**bold** or __bold__
***bold and italic*** or ___bold and italic___
~~strikethrough~~
```

- **Result**: *italic*, **bold**, ***bold and italic***, ~~strikethrough~~
- **Inline Code**

```
Use `backticks` for inline code
```

- **Result**: Use `backticks` for inline code
- **Tips**:
  - Use * or _ consistently throughout document
  - Avoid spaces between markers and text
  - Use inline code for *variable names*, *commands*, *file name*

# List

- **Unordered Lists:**

```
- Item 1
- Item 2
  - Nested item 2.1
  - Nested item 2.2
- Item 3


* You can also use asterisks
+ Or plus signs
```

# List

- **Ordered Lists:**

```
1. First item
2. Second item
3. Third item
    1. Nested item
    2. Another nested item
```

- **Task Lists**

```
- [x] Completed task
- [ ] Incomplete task
- [ ] Another task
```

# List

- **Tips:**
  - Indent nested lists with 2 or 4 spaces
  - Numbers don't have to be sequential (can all be 1.)
  - Blank line before list improves compatibility

# Links and Images

- **Links:**

```
[Link text](https://www.example.com)

[Link with title](https://www.example.com
"Hover text")


<https://www.example.com> (Automatic link)
```

- **Reference-Style Links:**

```
[Link text][reference]


[reference]: https://www.example.com "Optional
title"
```

# Links and Images

- **Images:**

```
![Alt text](image.jpg)
![Alt text](image.jpg "Image title")

![Alt text][image-ref]
[image-ref]: image.jpg
```

- **Tips:**
  - Always include alt text for accessibility
  - Use reference-style for repeated links
  - Relative paths work for local files

# Literate programming

- Idea from **Donald Knuth (1984)**
- Programs should be written to be **read by humans**, with explanation woven around the code
- A notebook is literate programming for data analysis: **question → code → result → interpretation**
- Code tells the computer what to do. The text tells the reader why.

# Same analysis, two notebooks

| **Code dump ☒** | **Literate notebook ☑** |
|---|---|
| 40 code cells, no text | Title, question, data source |
| `df2, df3, tmp` | `sales_2024, sales_by_city` |
| Charts without labels | Every chart has a title + takeaway |
| Reader must guess the conclusion | Findings stated in plain words |
| “Here is the chart, The chart shows sales” | “Sales in District 1 are **~2× higher** than any other district, but only on weekends” |

# A structure for analysis notebook

1. **Title** + author + date
2. **Question**: what are we trying to find out?
3. **Data**: source, link, what one row means
4. **Setup**: imports, seed, paths
5. **Sections**: one per sub-question, each ending with a **takeaway**
6. **Conclusion**: answer, limitations, next steps

# The README.md: your project's front door

- Project overview (and team info)
- Dataset source and description
- Research questions list
- Key findings summary
- File structure explanation (if multiple notebooks)
- How to run instructions
- Dependencies list

# Python

# Python

- Why did Guido van Rossum create Python programming language, although C had already existed at that time?
- Guido van Rossum: “I realized that the development of system administration utilities in C was taking too long”

# Why Python write faster

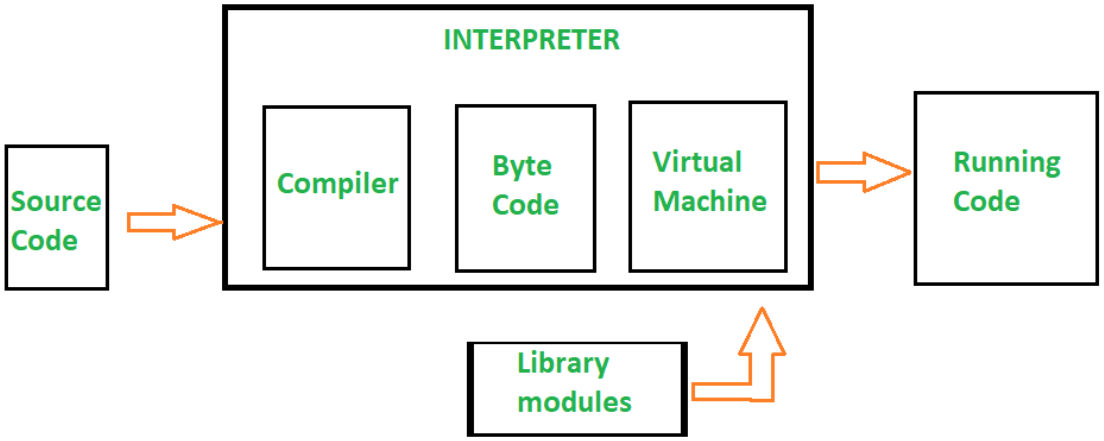
- **Interpreted language**: no need to compile all code before running; instead, programmers can code some lines of code and run immediately, and can continue to code and run from previous running state
- **Dynamic typing**

```
a = 1
...
a = 'hello'
```

```
def add_two_things(a, b):
    return a + b
# Can call add_two_things
# with 2 arbitrary objects
# having + operation
```

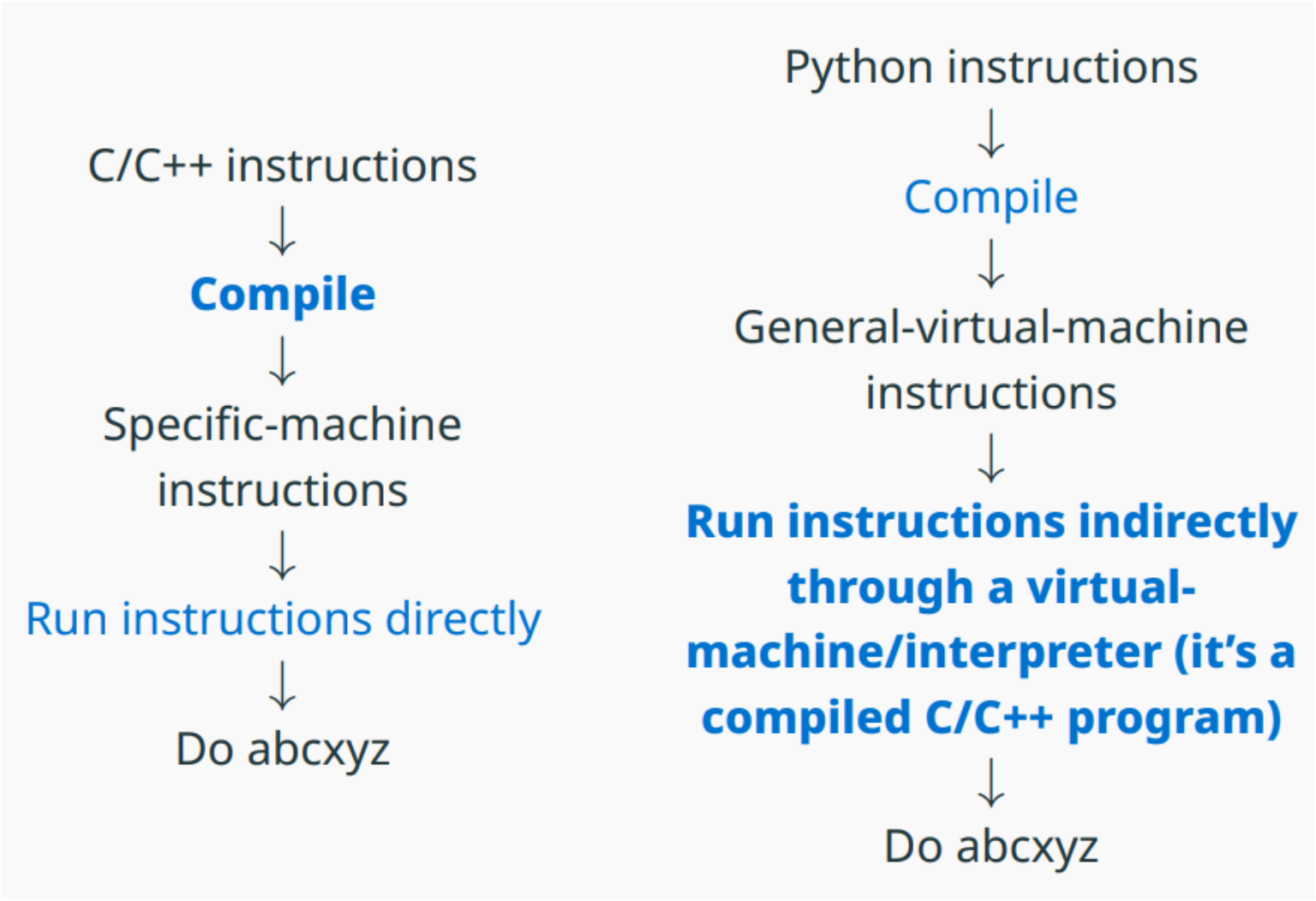
- **Automatic memory management**: programmers don't have to spend time doing malloc and free memories manually as in C/C++

# Why Python write faster






# Why Python write faster

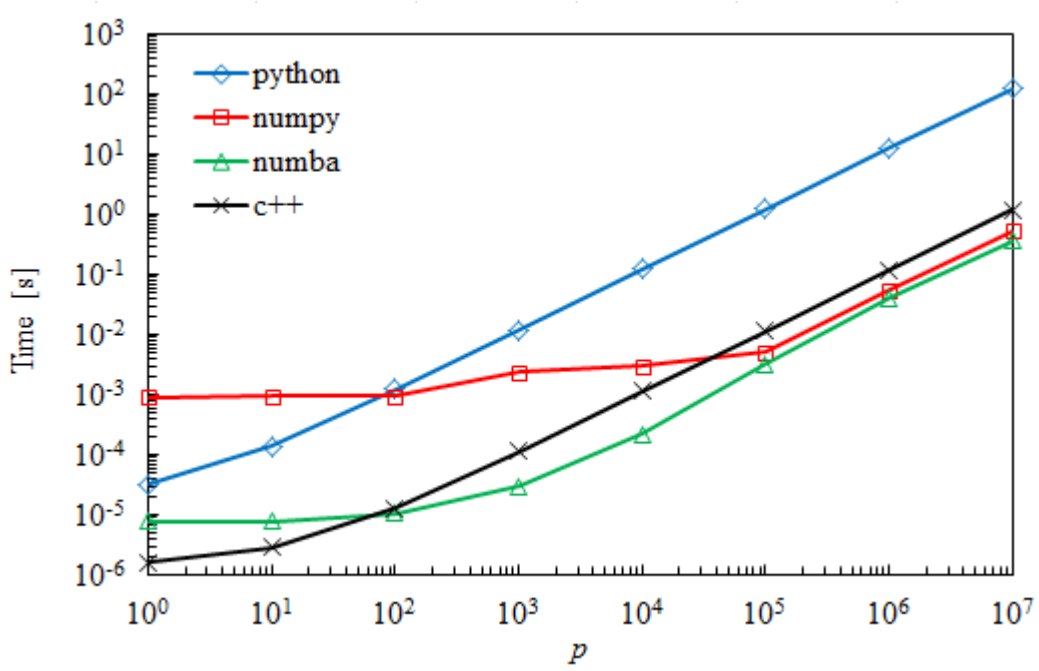


# Why Python run slower

- **Interpreted language**: interpreted languages *often run slower* than compiled languages, because in compiled languages, before running, compiler will look at all source code and do optimizations
- **Dynamic typing**: to achieve this, a variable in Python is just a pointer pointing to an object, and this object contains not only value but also meta info such as data type, …; when do an operation with 2 objects, Python interpreter first have to *spend time* opening these objects to identify data type
- **Automatic memory management**: it also costs *extra work and time* to know when to free allocated memories

# Homework

- Compare runtime of matrix multiplication program with: **C**, **Python**, **numpy**.
- Test with  several different size of matrix.
- Write report with Jupyter notebook with code and explain.






# THE END